PROJET D'ARCHITECTURE LOGICIELLE ET OBJETS RÉPARTIS

CAHIER DES CHARGES

GestActes

Gestion des actes administratifs via GitHub

Version	1.0
Date	Octobre 2026
Équipe	GUEMOGNE GERALDINE ESTHER JEUDIEU TSAGUE AUDREY DANIELLE KOUODZO ZANGUE REGINA LEUKAM TCHEUMALEU MAXIME METAMMO TEUMA DANIELA CLEDYA NDOUNGUE ESSOH ABEL THOMAS NOLACK KAWUNJIBI FRANGE PARKER NZEUTEM DOMOE EUNICE FELIXTINE TSEFACK CALVIN KLEIN VOUMIA OUALA DILANE
Encadrant	M. TCHUIGOUA CHARLU
Établissement	École Nationale Supérieure Polytechnique de Douala
Technologies	React (front-end) · Python / FastAPI (back-end) · GitHub (stockage et workflow)

Sommaire

1. Introduction et contexte 3
1.1 Contexte 3
1.2 Problématique 3
1.3 Objectifs du projet 3
1.4 Périmètre du projet 3
1.5 Glossaire 4
2. Acteurs et rôles 4
3. Besoins fonctionnels 5
3.1 Authentification et gestion des comptes 5
3.2 Gestion des demandes 5
3.3 Instruction et rédaction 6
3.4 Validation et signature 6
3.5 Recherche, historique et archivage 6
3.6 Administration 7
3.7 Notifications 7
4. Besoins non fonctionnels 7
5. Cas d'utilisation 8
5.1 Diagramme des cas d'utilisation 8
5.2 Description des cas d'utilisation principaux 9
6. Architecture du système 11
6.1 Principes directeurs 11
6.2 Vue d'ensemble des composants 11
6.3 Modélisation des actes dans GitHub 12
6.4 Workflow d'un acte 13
6.5 Diagramme de séquence : dépôt d'une demande et signature 15
6.6 Communication entre les services 15
6.7 Interfaces REST principales 15
6.8 Modèle de données 16
6.9 Architecture de déploiement 17
6.10 Sécurité 17
7. Choix technologiques 18
7.1 Front-end 18
7.2 Back-end 18
7.3 Infrastructure et outillage 19
8. Stratégie de tests et qualité 19
9. Planning et répartition des tâches 19
9.1 Planning prévisionnel 19
9.2 Répartition des rôles 20
10. Contraintes et risques 20
10.1 Contraintes 20
10.2 Analyse des risques 21
11. Livrables et critères de recette 21
11.1 Livrables attendus 21
11.2 Critères de recette 21
12. Conclusion 22
Annexe A : écriture d'un fichier dans GitHub 22
Annexe B : exemple de workflow GitHub Actions 23
Annexe C : structure proposée du code source 23

1. Introduction et contexte

1.1 Contexte

Les administrations produisent chaque jour un grand nombre d'actes administratifs : attestations, extraits d'actes d'état civil, décisions, certificats, autorisations. Leur traitement repose encore souvent sur des circuits papier ou sur des outils dispersés, ce qui rend le suivi difficile, allonge les délais et fragilise la traçabilité des décisions.

Un acte administratif a des besoins très proches de ceux d'un code source : il doit être **historisé** (qui a modifié quoi et quand), **relu et validé** par plusieurs personnes, **protégé** par des droits d'accès précis et **archivé** de façon fiable. Git et GitHub fournissent nativement ces mécanismes : commits horodatés, branches, Pull Requests, revues, protections de branche, automatisations avec GitHub Actions.

Ce projet, réalisé dans le cadre du module d'architecture logicielle et objets répartis, consiste à concevoir et développer **GestActes**, une application répartie qui s'appuie sur GitHub comme socle de stockage, de versionnage et de traçabilité des actes administratifs.

1.2 Problématique

Comment concevoir une application répartie, composée de services indépendants communicant à distance, qui gère le cycle de vie complet d'un acte administratif (demande, instruction, validation, signature, archivage) tout en garantissant sa traçabilité, son intégrité et son auditabilité grâce à GitHub ?

1.3 Objectifs du projet

Objectif général

Fournir une plateforme web permettant aux citoyens de demander des actes et aux agents de l'administration de les instruire, valider, signer et archiver, avec GitHub comme référentiel unique et auditable des actes.

Objectifs spécifiques

- Dématérialiser le circuit de traitement de trois types d'actes administratifs.
- Modéliser le cycle de vie d'un acte sous forme de workflow mappé sur les concepts GitHub (Issue, branche, Pull Request, merge, tag).
- Concevoir une architecture répartie en services faiblement couplés (authentification, actes, workflow, passerelle GitHub, notification).
- Garantir la traçabilité de chaque action : auteur, date, contenu avant et après modification.
- Offrir une interface web claire et adaptée à chaque rôle (citoyen, agent, validateur, administrateur).
- Documenter l'ensemble de la démarche : analyse, conception UML, choix technologiques, tests.

1.4 Périmètre du projet

Inclus dans le périmètre

- Trois types d'actes pris en charge dans la version 1 : **attestation de résidence**, **extrait d'acte de naissance** et **autorisation administrative simple**. D'autres types pourront être ajoutés via des modèles configurables.
- Gestion des comptes et des rôles, dépôt et suivi des demandes, instruction, validation, signature, archivage, recherche, historique, notifications par e-mail.
- Intégration avec l'API REST de GitHub et ses webhooks.

- Application web responsive en français.

Hors périmètre

- Paiement en ligne de timbres ou de frais administratifs.
- Signature électronique qualifiée au sens du règlement eIDAS : la signature est ici simulée par une empreinte SHA-256 horodatée et rattachée à l'identité du validateur.
- Interconnexion avec les registres officiels d'état civil : toutes les données manipulées sont **fictives**.
- Application mobile native et mode hors ligne.

1.5 Glossaire

Terme	Définition
Acte administratif	Document officiel émis par une administration, qui produit des effets juridiques ou atteste d'un fait (attestation, extrait, décision, autorisation).
Dépôt (repository)	Espace GitHub contenant les fichiers du projet et tout leur historique.
Commit	Enregistrement horodaté et signé d'un ensemble de modifications, avec son auteur et un message.
Branche	Ligne de développement parallèle permettant de travailler sur un brouillon sans toucher à la version publiée.
Issue	Ticket GitHub, utilisé ici pour représenter la demande d'un citoyen.
Pull Request (PR)	Demande de fusion d'une branche dans la branche principale, soumise à relecture et approbation.
GitHub Actions	Service d'automatisation de GitHub exécutant des tâches déclenchées par des événements (génération du PDF, contrôles).
Webhook	Appel HTTP envoyé par GitHub à notre application lorsqu'un événement survient (PR fusionnée, revue soumise).
API REST	Interface d'accès à un service via des requêtes HTTP (GET, POST, PUT, DELETE) échangeant du JSON.
JWT	JSON Web Token : jeton signé transportant l'identité et le rôle d'un utilisateur authentifié.
RBAC	Role-Based Access Control : contrôle d'accès fondé sur les rôles.
Message broker	Intermédiaire (RabbitMQ) qui achemine des messages de façon asynchrone entre services.
SPA	Single Page Application : application web chargée une fois et mise à jour dynamiquement (React).

2. Acteurs et rôles

Quatre profils d'utilisateurs interagissent avec le système, auxquels s'ajoutent deux systèmes externes.

Acteur	Description	Droits principaux
Citoyen	Personne qui sollicite un acte auprès de l'administration.	Créer un compte, déposer une demande, joindre des pièces, suivre l'état de ses demandes, télécharger ses actes signés.
Agent	Employé chargé d'instruire les demandes et de rédiger les actes.	Consulter la file des demandes, prendre en charge, rédiger et modifier un brouillon, demander un complément, soumettre à validation.
Validateur	Responsable hiérarchique habilité à valider et signer.	Consulter les actes en attente avec leurs différences, approuver, rejeter ou demander des corrections, signer.
Administrateur	Responsable technique et fonctionnel de la plateforme.	Gérer les utilisateurs et rôles, configurer les modèles d'actes, consulter le journal d'audit et les statistiques.
GitHub (système externe)	Plateforme d'hébergement des dépôts et du workflow.	Stocke les actes, expose l'API REST, émet des webhooks, exécute les Actions.
Serveur SMTP (système externe)	Service d'envoi d'e-mails.	Distribue les notifications aux utilisateurs.

3. Besoins fonctionnels

Les exigences sont identifiées par un code (EF-xx) et classées par priorité : **Haute** (indispensable à la version minimale), **Moyenne** (souhaitable) et **Basse** (optionnelle).

3.1 Authentification et gestion des comptes

ID	Besoin	Priorité	Acteur
EF-01	Créer un compte citoyen (nom, prénom, e-mail, mot de passe).	Haute	Citoyen
EF-02	Se connecter et se déconnecter de façon sécurisée (jeton JWT à durée limitée et jeton de rafraîchissement).	Haute	Tous
EF-03	Appliquer un contrôle d'accès par rôle sur toutes les fonctionnalités.	Haute	Système
EF-04	Réinitialiser son mot de passe par e-mail.	Moyenne	Tous

3.2 Gestion des demandes

ID	Besoin	Priorité	Acteur
EF-05	Consulter le catalogue des types d'actes disponibles.	Haute	Citoyen
EF-06	Remplir un formulaire dynamique généré à partir du modèle de l'acte et joindre des pièces justificatives (PDF, JPG, 5 Mo maximum).	Haute	Citoyen
EF-07	Créer automatiquement une Issue GitHub et le dossier de l'acte à chaque nouvelle demande.	Haute	Système
EF-08	Suivre l'état et l'historique de ses demandes.	Haute	Citoyen
EF-09	Annuler une demande tant qu'elle n'est pas prise en charge.	Moyenne	Citoyen

ID	Besoin	Priorité	Acteur
EF-10	Télécharger l'acte signé au format PDF.	Haute	Citoyen

3.3 Instruction et rédaction

ID	Besoin	Priorité	Acteur
EF-11	Consulter la file des demandes à traiter avec filtres (type, date, statut).	Haute	Agent
EF-12	Prendre en charge une demande : assignation de l'Issue et création d'une branche dédiée.	Haute	Agent
EF-13	Générer le brouillon de l'acte à partir du modèle et des données saisies par le citoyen.	Haute	Agent
EF-14	Modifier le brouillon ; chaque enregistrement produit un commit.	Haute	Agent
EF-15	Demander un complément d'information ou une pièce manquante au citoyen.	Moyenne	Agent
EF-16	Soumettre l'acte à validation (ouverture d'une Pull Request).	Haute	Agent

3.4 Validation et signature

ID	Besoin	Priorité	Acteur
EF-17	Consulter les actes en attente de validation avec les différences par rapport à la version précédente.	Haute	Validateur
EF-18	Approuver, rejeter ou demander des corrections avec un commentaire motivé obligatoire en cas de rejet.	Haute	Validateur
EF-19	Signer l'acte : fusion de la PR, calcul de l'empreinte SHA-256, horodatage et identité du signataire.	Haute	Validateur
EF-20	Générer automatiquement le PDF de l'acte avec numéro unique et QR code de vérification (GitHub Actions).	Moyenne	Système
EF-21	Vérifier l'authenticité d'un acte à partir de son numéro ou de son QR code.	Basse	Public

3.5 Recherche, historique et archivage

ID	Besoin	Priorité	Acteur
EF-22	Rechercher des actes par numéro, type, date, statut ou demandeur.	Haute	Agent, Validateur
EF-23	Consulter l'historique complet d'un acte (liste des commits, auteurs, dates, différences).	Haute	Agent, Validateur
EF-24	Archiver un acte signé (tag Git et copie dans le dépôt d'archives).	Moyenne	Système
EF-25	Exporter une liste d'actes au format CSV.	Basse	Agent, Validateur

3.6 Administration

ID	Besoin	Priorité	Acteur
EF-26	Créer, modifier, désactiver des utilisateurs et leur attribuer des rôles.	Haute	Administrateur
EF-27	Gérer les modèles d'actes (schéma des champs, gabarit du document).	Moyenne	Administrateur
EF-28	Consulter le journal d'audit (qui a fait quoi, quand).	Moyenne	Administrateur
EF-29	Visualiser un tableau de bord : nombre d'actes par statut, délais moyens de traitement.	Basse	Administrateur

3.7 Notifications

ID	Besoin	Priorité	Acteur
EF-30	Envoyer un e-mail à chaque changement d'état d'une demande (prise en charge, complément demandé, rejet, signature).	Moyenne	Système
EF-31	Afficher des notifications dans l'interface (cloche avec compteur).	Basse	Tous

4. Besoins non fonctionnels

Catégorie	Exigence	Critère de vérification
Sécurité	Authentification par JWT, mots de passe hachés (bcrypt), communications chiffrées (HTTPS), contrôle d'accès par rôle, validation stricte des entrées, secrets stockés en variables d'environnement, jeton GitHub aux droits minimaux.	Aucun accès à une ressource sans jeton valide ; tests d'autorisation par rôle ; aucun secret dans le dépôt de code.
Intégrité et traçabilité	Toute modification d'un acte produit un commit identifiant l'auteur ; les actes signés ne sont plus modifiables (branche principale protégée).	Historique Git complet et cohérent avec le journal d'audit.
Performance	Interface réactive ; les appels à GitHub sont mis en cache ou rendus asynchrones.	95 % des lectures en moins de 500 ms (hors appel GitHub) ; création d'une demande en moins de 3 s.
Disponibilité et résilience	En cas d'indisponibilité de GitHub, les opérations d'écriture sont mises en file d'attente et rejouées avec reprise progressive.	Aucune perte de demande lors d'une coupure simulée de GitHub.
Scalabilité	Services sans état, conteneurisés, pouvant être dupliqués indépendamment.	Démarrage de plusieurs instances d'un même service sans changement de code.
Interopérabilité	API REST documentée selon OpenAPI 3 ; échanges en JSON.	Documentation interactive (Swagger UI) générée et à jour.
Maintenabilité	Code structuré par services, typage (Pydantic, TypeScript), outils de lint, intégration continue.	Couverture de tests supérieure à 70 % sur les services ; pipeline CI au vert.

Catégorie	Exigence	Critère de vérification
Utilisabilité	Interface en français, responsive, messages d'erreur clairs, accessibilité de base (contrastes, navigation clavier, libellés).	Parcours principal réalisable en moins de 5 clics ; test sur mobile et ordinateur.
Portabilité	Déploiement complet par Docker Compose.	Application lancée par une seule commande sur une machine vierge.
Conformité	Minimisation des données, données fictives uniquement, dépôts privés ; limites du droit à l'effacement avec Git documentées.	Revue de conformité jointe au dossier.
Contraintes de l'API GitHub	Respect des quotas (5 000 requêtes par heure en mode authentifié) : requêtes conditionnelles (ETag), cache, webhooks plutôt qu'interrogation périodique.	Consommation de quota mesurée et inférieure à 50 % en usage normal.

5. Cas d'utilisation

5.1 Diagramme des cas d'utilisation

Le diagramme ci-dessous présente les principales fonctionnalités du système regroupées par acteur.

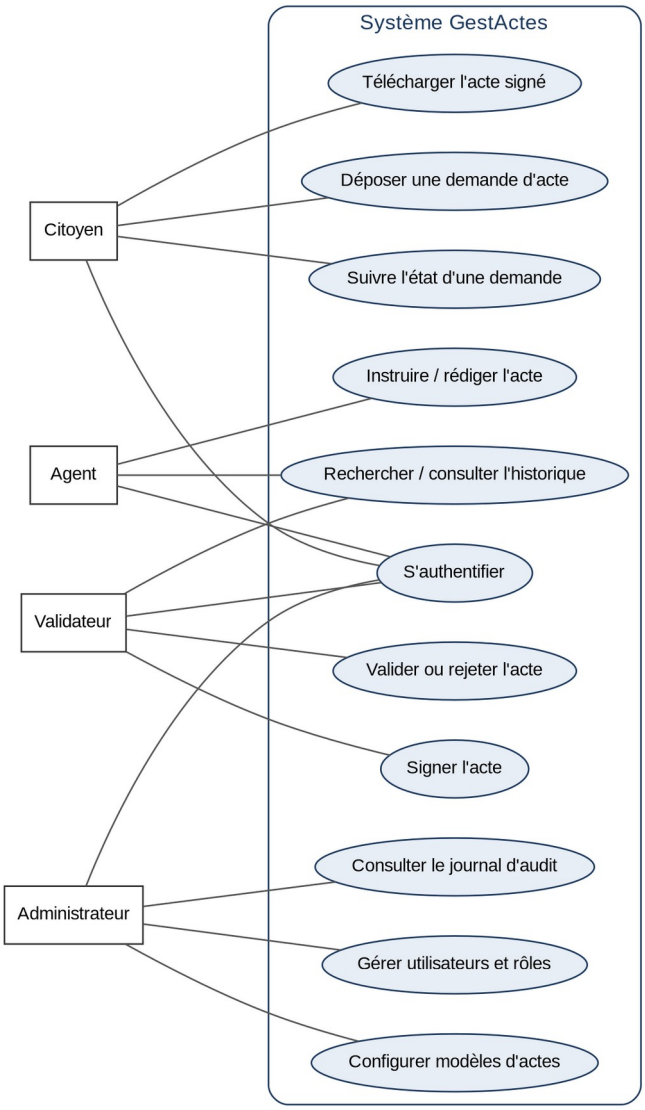


Figure 1 : diagramme des cas d'utilisation de GestActes

5.2 Description des cas d'utilisation principaux

UC-02 : Déposer une demande d'acte	
Acteur principal	Citoyen
Préconditions	Le citoyen est authentifié et son compte est actif.
Scénario nominal	1. Le citoyen choisit un type d'acte dans le catalogue. 2. Le système affiche le formulaire correspondant au modèle. 3. Le citoyen renseigne les champs et joint ses pièces. 4. Le système valide les données et enregistre la demande. 5. Le système crée l'Issue GitHub et le dossier de l'acte, puis attribue un numéro unique. 6. Le système confirme le dépôt et envoie un e-mail de confirmation.
Scénarios alternatifs	3a. Données invalides ou pièce trop volumineuse : le système signale l'erreur et reste sur le formulaire. 5a. GitHub indisponible : la demande est enregistrée localement et sa création dans GitHub est mise en file d'attente.
Postconditions	Une demande à l'état « Demandée » existe, avec une Issue GitHub associée.

UC-05 : Instruire et rédiger l'acte	
Acteur principal	Agent
Préconditions	L'agent est authentifié ; une demande à l'état « Demandée » existe.
Scénario nominal	1. L'agent ouvre la file des demandes et sélectionne une demande. 2. Il la prend en charge ; le système crée la branche acte/<numéro> et assigne l'Issue. 3. Le système génère le brouillon à partir du modèle et des données du citoyen. 4. L'agent complète et corrige le brouillon ; chaque enregistrement crée un commit. 5. L'agent soumet l'acte à validation ; le système ouvre la Pull Request.
Scénarios alternatifs	4a. Pièce manquante : l'agent demande un complément ; le citoyen est notifié et la demande reste en instruction.
Postconditions	L'acte est à l'état « En validation » avec une Pull Request ouverte.

UC-06 : Valider ou rejeter un acte	
Acteur principal	Validateur
Préconditions	Le validateur est authentifié ; une Pull Request est ouverte pour l'acte.
Scénario nominal	1. Le validateur consulte l'acte et les différences avec la version précédente. 2. Il approuve l'acte ; le système enregistre la revue d'approbation sur la Pull Request. 3. L'acte passe à l'état « Validée ».
Scénarios alternatifs	2a. Corrections demandées : revue « request changes » avec commentaire ; l'acte retourne en instruction. 2b. Rejet : commentaire motivé obligatoire ; la PR est fermée sans fusion ; le citoyen est notifié.
Postconditions	L'acte est « Validée », « En instruction » ou « Rejetée » selon la décision.

UC-07 : Signer l'acte	
Acteur principal	Validateur
Préconditions	L'acte est à l'état « Validée ».
Scénario nominal	1. Le validateur confirme la signature. 2. Le système calcule l'empreinte SHA-256 du contenu, l'horodate et l'associe à l'identité du signataire. 3. Le système fusionne la PR dans la branche principale. 4. GitHub Actions génère le PDF avec numéro et QR code, puis pose le tag d'archivage. 5. Le citoyen est notifié et peut télécharger l'acte.
Scénarios alternatifs	3a. Échec de fusion : le système annule la signature, journalise l'erreur et informe le validateur.
Postconditions	L'acte est « Signée » puis « Archivée » ; il n'est plus modifiable.

6. Architecture du système

6.1 Principes directeurs

- **Architecture orientée services** : chaque service a une responsabilité unique, son propre code et son propre conteneur.
- **GitHub comme source de vérité** du contenu des actes ; la base PostgreSQL ne conserve que les comptes, les métadonnées et un index de recherche.
- **Communication hybride** : appels REST synchrones pour les requêtes utilisateur, messages asynchrones (RabbitMQ) pour les événements et notifications.
- **Services sans état** et configuration par variables d'environnement, pour faciliter la duplication et le déploiement.
- **Sécurité en profondeur** : authentification à la passerelle, autorisation dans chaque service, secrets isolés.

6.2 Vue d'ensemble des composants

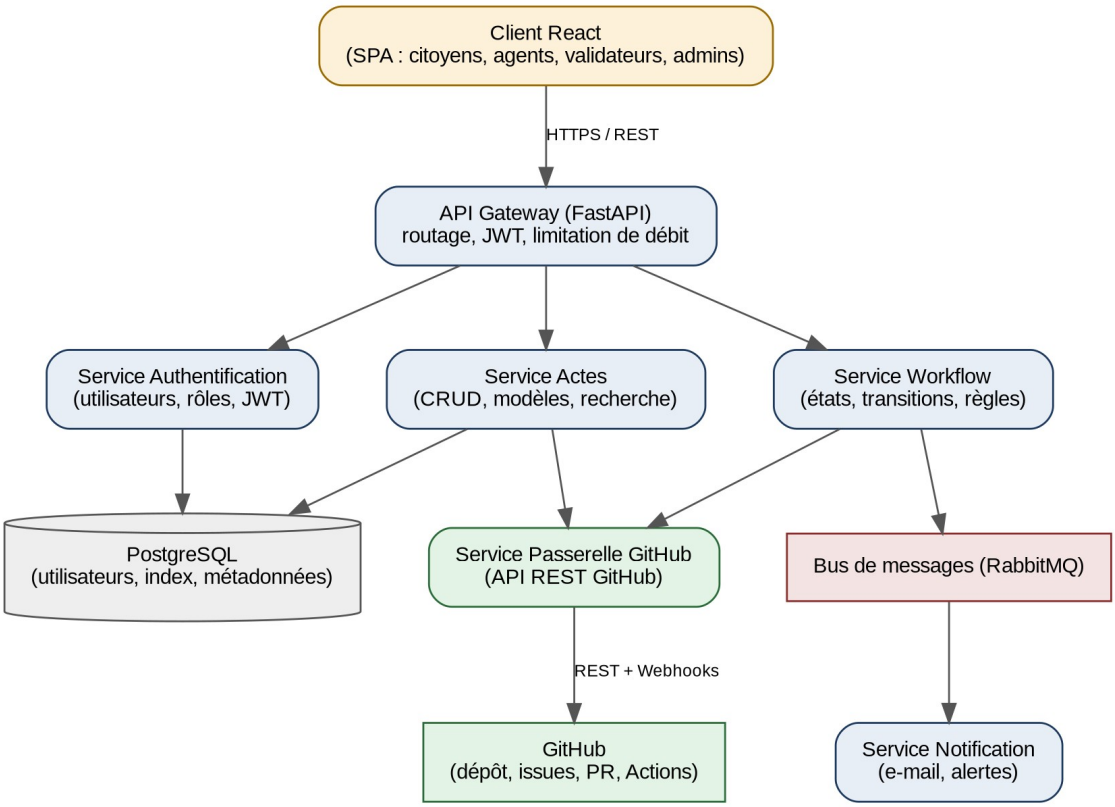


Figure 2 : architecture générale (diagramme de composants)

Composant	Responsabilités	Technologie
Client web	Interfaces par rôle, formulaires dynamiques, suivi des demandes, relecture des différences.	React, TypeScript
API Gateway	Point d'entrée unique, vérification du JWT, routage vers les services, limitation de débit, CORS.	Python, FastAPI, httpx
Service Authentification	Comptes, mots de passe, rôles, émission et rafraîchissement des jetons.	Python, FastAPI, SQLAlchemy

Composant	Responsabilités	Technologie
Service Actes	Catalogue des types d'actes, modèles, création et lecture des actes, recherche, pièces jointes.	Python, FastAPI, SQLAlchemy
Service Workflow	Machine à états des actes, règles de transition par rôle, publication des événements, traitement des webhooks GitHub.	Python, FastAPI, aio-pika
Passerelle GitHub	Seul composant qui dialogue avec GitHub : commits, Issues, branches, PR, revues, tags ; gestion des quotas, du cache et des reprises.	Python, httpx (ou PyGithub)
Service Notification	Consomme les événements et envoie les e-mails et notifications internes.	Python, aio-pika, SMTP
Bus de messages	Acheminement asynchrone et fiable des événements entre services.	RabbitMQ
Base de données	Utilisateurs, métadonnées des demandes, index de recherche, journal d'audit.	PostgreSQL

6.3 Modélisation des actes dans GitHub

Chaque concept métier est associé à un concept GitHub. Le backend utilise un compte technique (idéalement une GitHub App) aux droits limités ; l'identité réelle de l'agent ou du validateur est portée par l'auteur du commit et par les métadonnées de la Pull Request.

Élément métier	Concept GitHub	Détail d'implémentation
Stockage des actes	Dépôt privé actes-data	Un dossier par acte : actes/<type>/<année>/<numéro>/
Archives	Dépôt privé actes-archives	Copie en lecture seule des actes signés, protégée contre toute modification
Contenu de l'acte	Fichiers	acte.json (données), acte.md (gabarit rendu), pieces/ (justificatifs), acte.pdf (après signature)
Demande du citoyen	Issue	Titre normalisé ; étiquettes de statut (statut:demandée, statut:en-instruction, etc.)
Brouillon	Branche	acte/<numéro>, une branche par acte
Modification	Commit	Message normalisé : [ACTE-2026-00042] instruction : correction de l'adresse
Soumission à validation	Pull Request	Revue demandée aux validateurs (équipe GitHub ou CODEOWNERS)
Décision	Revue de PR	Approve, Request changes, ou fermeture de la PR en cas de rejet
Signature	Fusion (merge) dans main	Branche main protégée : fusion uniquement après approbation
Archivage	Tag	acte-2026-00042-v1 posé après génération du PDF
Automatisations	GitHub Actions	Validation du JSON, génération du PDF et du QR code, calcul d'empreinte

Élément métier	Concept GitHub	Détail d'implémentation
Retour d'événements	Webhooks	PR fusionnée, revue soumise, Issue fermée : transmis au service Workflow

Arborescence du dépôt de données

```
actes-data/
  actes/
    attestation-residence/
      2026/
        ACTE-2026-00042/
          acte.json
          acte.md
          pieces/
            justificatif-domicile.pdf
    extrait-naissance/
    autorisation-administrative/
  modeles/
    attestation-residence.schema.json
    attestation-residence.template.md
  .github/
    workflows/generer-pdf.yml
    CODEOWNERS
```

Exemple de fichier acte.json

```
{
  "numero": "ACTE-2026-00042",
  "type": "attestation-residence",
  "statut": "en-validation",
  "demandeur": { "id": 17, "nom": "Ndiaye", "prenom": "Awa" },
  "donnees": {
    "adresse": "12 rue des Lilas, 75000 Ville",
    "residence_depuis": "2021-03-01"
  },
  "instruction": { "agent_id": 4, "date_prise_en_charge": "2026-10-12" },
  "signature": null
}
```

6.4 Workflow d'un acte

Le cycle de vie d'un acte est une machine à états gérée par le service Workflow. Une transition n'est possible que si l'utilisateur possède le rôle requis et si l'état de départ est correct ; chaque transition déclenche l'action GitHub correspondante.

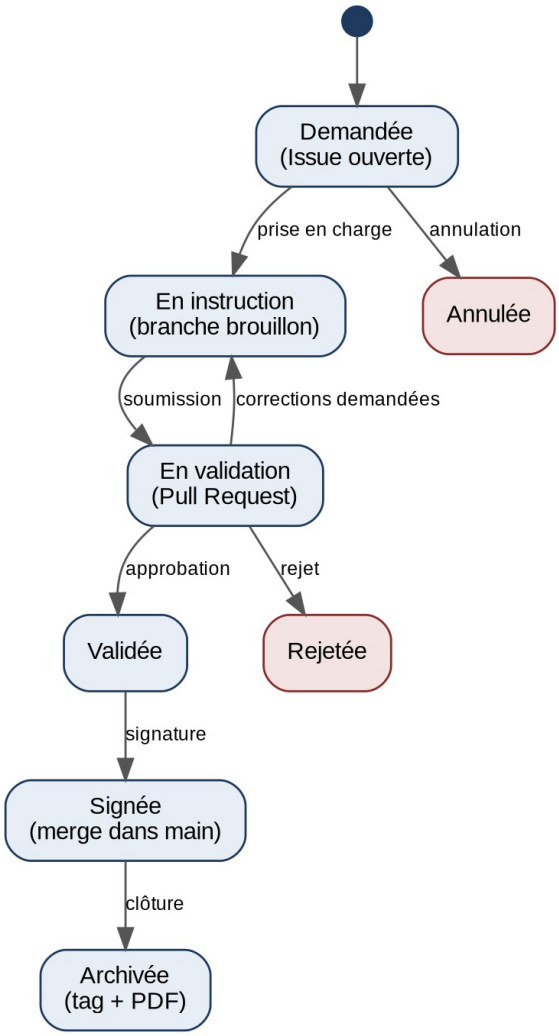


Figure 3 : diagramme d'états d'un acte

État de départ	Événement	État d'arrivée	Acteur	Action dans GitHub
Demandée	Prise en charge	En instruction	Agent	Création de la branche, assignation de l'Issue, étiquette de statut
Demandée	Annulation	Annulée	Citoyen	Fermeture de l'Issue avec l'étiquette annulée
En instruction	Soumission	En validation	Agent	Dernier commit, ouverture de la Pull Request
En validation	Corrections demandées	En instruction	Validateur	Revue « Request changes »
En validation	Rejet	Rejetée	Validateur	Fermeture de la PR sans fusion, fermeture de l'Issue
En validation	Approbation	Validée	Validateur	Revue « Approve »
Validée	Signature	Signée	Validateur	Fusion de la PR dans main, empreinte SHA-256
Signée	Clôture automatique	Archivée	Système	GitHub Action : PDF et QR code, tag, copie dans actes-archives, fermeture de l'Issue

6.5 Diagramme de séquence : dépôt d'une demande et signature

Le scénario ci-dessous illustre la collaboration entre les services lors du dépôt d'une demande (messages 1 à 8) puis, plus tard, de la signature de l'acte détectée grâce au webhook GitHub (messages 9 et 10).

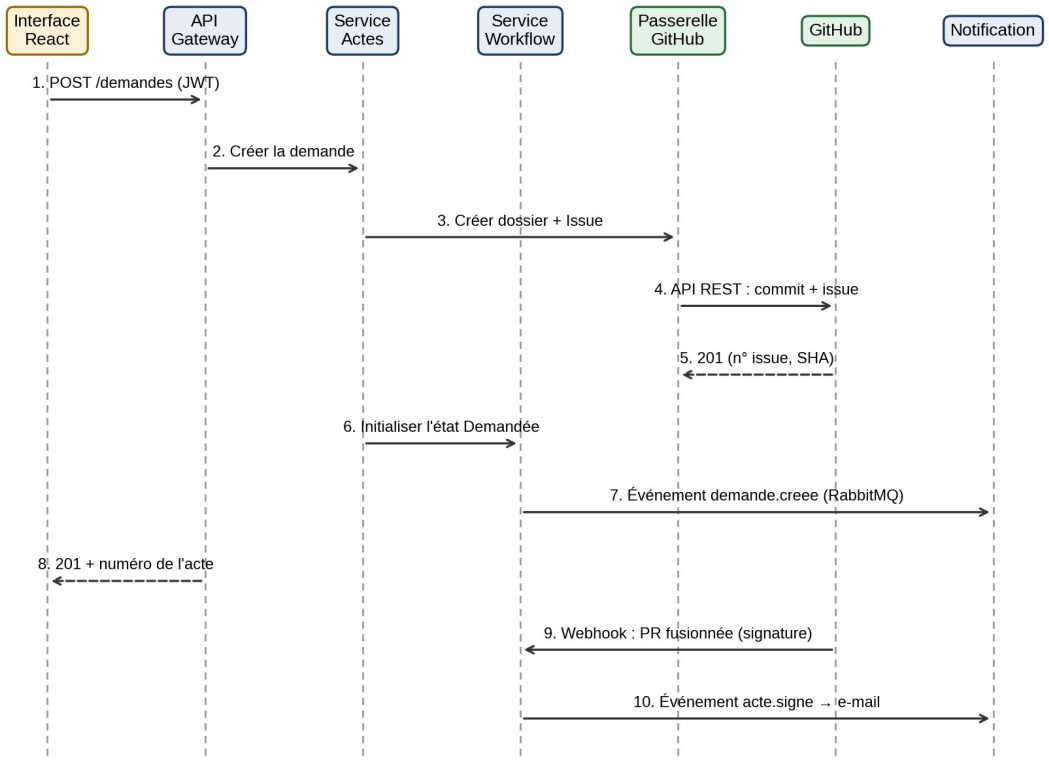


Figure 4 : diagramme de séquence du dépôt et de la signature d'un acte

6.6 Communication entre les services

- **REST synchrone** (HTTP/JSON) : du client vers la passerelle, puis de la passerelle vers les services et entre services lorsqu'une réponse immédiate est nécessaire.
- **Messagerie asynchrone** (RabbitMQ) : le service Workflow publie des événements (demande.creee, acte.soumis, acte.valide, acte.rejete, acte.signe, acte.archive) consommés notamment par le service Notification. Les messages sont persistants et acquittés.
- **Webhooks** : GitHub appelle l'endpoint POST /webhooks/github du service Workflow. L'authenticité est vérifiée par la signature HMAC-SHA256 de la requête et le traitement est idempotent.
- **Résilience** : délais d'attente (timeouts), nouvelles tentatives avec attente croissante pour les appels à GitHub, file d'attente de secours en cas d'indisponibilité.

6.7 Interfaces REST principales

Méthode	Route	Description	Rôle autorisé
POST	/auth/register	Créer un compte citoyen	Public
POST	/auth/login	Obtenir un jeton d'accès et de rafraîchissement	Public
POST	/auth/refresh	Renouveler le jeton d'accès	Connecté

Méthode	Route	Description	Rôle autorisé
GET	/auth/me	Profil de l'utilisateur courant	Connecté
GET	/types-actes	Lister les types d'actes et leurs modèles	Connecté
POST	/demandes	Déposer une demande	Citoyen
GET	/demandes	Lister les demandes (filtres, pagination)	Selon le rôle
GET	/demandes/{id}	Détail d'une demande	Selon le rôle
DELETE	/demandes/{id}	Annuler une demande	Citoyen
POST	/actes/{id}/prise-en-charge	Prendre en charge une demande	Agent
PUT	/actes/{id}/contenu	Modifier le brouillon (crée un commit)	Agent
POST	/actes/{id}/soumission	Soumettre à validation (ouvre la PR)	Agent
POST	/actes/{id}/decision	Approuver, rejeter ou demander des corrections	Validateur
POST	/actes/{id}/signature	Signer l'acte	Validateur
GET	/actes/{id}/historique	Historique des commits et événements	Agent, Validateur
GET	/actes/{id}/pdf	Télécharger le PDF signé	Demandeur, Agent, Validateur
GET	/actes?q=	Recherche multicritère	Agent, Validateur
GET	/verification/{numero}	Vérifier l'authenticité d'un acte	Public
GET	/admin/utilisateurs	Gérer les utilisateurs (CRUD complet)	Administrateur
GET	/admin/audit	Consulter le journal d'audit	Administrateur
POST	/webhooks/github	Réception des événements GitHub	GitHub (HMAC)

6.8 Modèle de données

La base PostgreSQL conserve les données de gestion et un index ; le contenu faisant foi des actes reste dans GitHub. Une tâche de réconciliation périodique compare l'index à l'état des dépôts.

Entité	Attributs principaux
Utilisateur	id, nom, prénom, e-mail (unique), mot_de_passe_haché, rôle, actif, créé_le
TypeActe	id, code, libellé, schéma_json, modèle_document, actif
Demande	id, numéro (unique), type_acte_id, demandeur_id, statut, issue_github, branche, pr_github, créé_le, mis_à_jour_le
Assignation	id, demande_id, utilisateur_id, rôle (agent ou validateur), date
PieceJointe	id, demande_id, nom, chemin_dans_le_dépôt, empreinte_sha256, taille
EvenementAudit	id, demande_id, acteur_id, action, détails, sha_commit, horodatage
Notification	id, destinataire_id, message, lue, créée_le

Relations : un Utilisateur dépose plusieurs Demandes ; une Demande appartient à un TypeActe, possède plusieurs PiecesJointes, Assignations et EvenementsAudit ; un Utilisateur reçoit plusieurs Notifications.

6.9 Architecture de déploiement

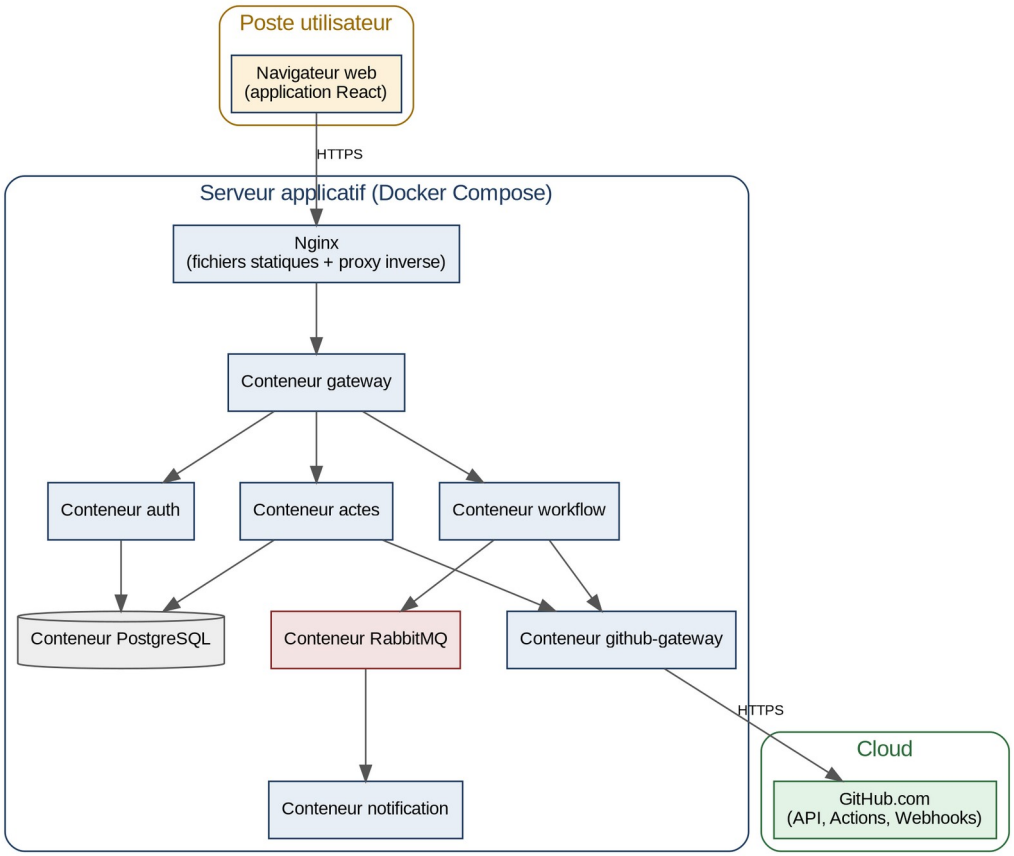


Figure 5 : diagramme de déploiement

Tous les composants sont packagés en conteneurs Docker et orchestrés par Docker Compose. Nginx sert les fichiers statiques de l'application React et joue le rôle de proxy inverse vers la passerelle. Seuls les ports de Nginx sont exposés ; les autres services communiquent sur un réseau interne. Les variables sensibles (clé de signature JWT, jeton GitHub, secret de webhook, identifiants de base) sont injectées par fichier d'environnement non versionné.

6.10 Sécurité

- **Authentification** : mots de passe hachés avec bcrypt ; jetons d'accès de courte durée (30 minutes) et jetons de rafraîchissement renouvelables.
- **Autorisation** : contrôle par rôle (RBAC) appliqué à la passerelle puis revérifié dans chaque service ; un citoyen ne peut accéder qu'à ses propres demandes.
- **Transport** : HTTPS de bout en bout en production, en-têtes de sécurité (HSTS, CSP) et CORS restreint à l'origine du front-end.
- **Données** : validation des entrées avec Pydantic, requêtes paramétrées via l'ORM, contrôle du type et de la taille des fichiers téléversés, chiffrement des champs sensibles avant écriture dans GitHub.
- **GitHub** : dépôts privés, branche main protégée (revue obligatoire, pas de push direct), jeton aux droits minimaux, secret de webhook vérifié par HMAC.

- **Audit** : chaque action sensible est journalisée avec son auteur, son horodatage et le SHA du commit associé.

7. Choix technologiques

7.1 Front-end

Besoin	Choix	Justification
Bibliothèque UI	React 18 avec TypeScript	Composants réutilisables, écosystème riche, typage statique qui limite les erreurs.
Outil de build	Vite	Démarrage rapide, rechargement à chaud, configuration minimale.
Navigation	React Router	Routage côté client et routes protégées par rôle.
Données serveur	TanStack Query et Axios	Cache, rafraîchissement automatique, gestion des états de chargement et d'erreur.
Formulaires	React Hook Form et Zod	Formulaires dynamiques performants et validation par schéma.
Composants visuels	Material UI (ou Tailwind CSS)	Interface cohérente, responsive et accessible dès le départ.
Tests	Vitest, React Testing Library, Playwright	Tests unitaires, de composants et de bout en bout.

7.2 Back-end

Besoin	Choix	Justification
Langage	Python 3.12	Lisibilité, productivité, nombreuses bibliothèques pour GitHub, PDF et messagerie.
Framework web	FastAPI	Hautes performances (asynchrone), validation automatique, documentation OpenAPI générée.
Validation et schémas	Pydantic v2	Contrats de données explicites entre services.
Accès aux données	SQLAlchemy 2 et Alembic	ORM mature et migrations versionnées du schéma.
Base de données	PostgreSQL 16	Fiabilité, transactions, recherche textuelle intégrée.
Client GitHub	httpx (ou PyGithub)	Appels REST asynchrones, gestion des délais et des nouvelles tentatives.
Messagerie	RabbitMQ avec aio-pika	Distribution fiable des événements, files persistantes.
Sécurité	PyJWT, passlib (bcrypt)	Standards éprouvés pour les jetons et le hachage.
Génération PDF	ReportLab ou WeasyPrint, qrcode	Production du PDF final et du QR code de vérification dans GitHub Actions.
Tests	pytest, httpx TestClient, respx	Tests unitaires et d'intégration avec GitHub simulé.

7.3 Infrastructure et outillage

Besoin	Choix	Justification
Conteneurisation	Docker et Docker Compose	Environnement reproductible, lancement en une commande.
Serveur web	Nginx	Service des fichiers statiques et proxy inverse.
Intégration continue	GitHub Actions	Lint, tests et construction des images à chaque push.
Qualité du code	Ruff, mypy, ESLint, Prettier	Style homogène et détection précoce des erreurs.
Conception	UML (PlantUML ou draw.io), Figma	Diagrammes et maquettes versionnés avec le projet.
Gestion de projet	GitHub Projects	Suivi des tâches au plus près du code.

8. Stratégie de tests et qualité

Niveau	Objectif	Outils
Tests unitaires	Valider la logique métier (machine à états, règles de rôles, validation des modèles).	pytest, Vitest
Tests d'intégration	Vérifier l'interaction entre services et avec une API GitHub simulée.	pytest, respx, base de test
Tests de contrat	S'assurer que les services respectent leur spécification OpenAPI.	Schémathèque OpenAPI, Pydantic
Tests de bout en bout	Parcourir le scénario complet de la demande à la signature depuis l'interface.	Playwright
Tests de sécurité	Contrôler les accès par rôle, l'expiration des jetons, les entrées malveillantes.	pytest, revue manuelle

- Chaque Pull Request du projet est relue par au moins un autre membre de l'équipe et ne peut être fusionnée que si le pipeline CI est au vert.
- Convention de commits (Conventional Commits) et branches courtes par fonctionnalité.
- Un jeu de données de démonstration (utilisateurs, modèles, demandes fictives) est fourni pour la démonstration de l'application.

9. Planning et répartition des tâches

9.1 Planning prévisionnel

Le planning ci-dessous est proposé sur huit semaines et devra être ajusté au calendrier imposé par l'enseignant. Les fonctionnalités de priorité Haute constituent la version minimale (MVP) à livrer en priorité.

Phase	Période	Contenu	Livrable
1. Cadrage et analyse	Semaine 1	Compréhension du sujet, périmètre, acteurs, besoins, cas d'utilisation	Cahier des charges validé
2. Conception	Semaine 2	Architecture détaillée, workflow, modèle de données, contrat OpenAPI, maquettes	Dossier de conception UML et maquettes
3. Socle technique	Semaine 3	Docker Compose, passerelle, service d'authentification, squelette React, CI	Environnement fonctionnel, connexion opérationnelle
4. Passerelle GitHub et Actes	Semaine 4	Prototype de création de fichier et d'Issue via l'API, service Actes, modèles	Dépôt d'une demande jusqu'à GitHub
5. Workflow et messagerie	Semaine 5	Machine à états, branches et PR, RabbitMQ, webhooks	Cycle de vie complet côté API
6. Interfaces	Semaine 6	Écrans citoyen, agent, validateur ; service Notification	Parcours complet via l'interface
7. Compléments et tests	Semaine 7	Administration, PDF et QR code, recherche, tests, sécurité	Application stabilisée, rapport de tests
8. Recette et finalisation	Semaine 8	Corrections, documentation, jeu de démonstration	Application et dossier finaux

9.2 Répartition des rôles

Rôle	Responsabilités	Membre
Chef de projet	Coordination, planning, suivi des livrables, rédaction du cahier des charges	NZEUTEM DOMOE EUNICE FELIXTINE
Architecte	Architecture, workflow, contrats d'API, revue technique	NDOUNGUE ESSOH ABEL THOMAS
Développeur front-end	Application React, intégration des maquettes, tests d'interface	GUEMOGNE GERALDINE ESTHER JEUDIEU TSAGUE AUDREY DANIELLE
Développeur back-end (services)	Authentification, Actes, Workflow, base de données	KOUODZO ZANGUE REGINA LEUKAM TCHEUMALEU MAXIME METAMMO TEUMA DANIELA CLEDYA
Développeur intégration	Passerelle GitHub, webhooks, GitHub Actions, RabbitMQ	NOLACK KAWUNJIBI FRANGE PARKER TSEFACK CALVIN KLEIN
Qualité et documentation	Tests, intégration continue, documentation, jeu de démonstration	VOUMIA OUALA DILANE

10. Contraintes et risques

10.1 Contraintes

- **Techniques** : React pour le front-end, Python pour le back-end, GitHub (API REST) comme socle de stockage et de workflow.

- **Pédagogiques** : mise en œuvre d'une architecture répartie avec communication à distance entre composants.
- **Temporelles** : livraison du cahier des charges puis de l'application aux dates fixées par l'enseignant.
- **Réglementaires** : usage exclusif de données fictives ; une exploitation réelle exigerait un hébergement conforme (RGPD, souveraineté des données).

10.2 Analyse des risques

Risque	Probabilité	Impact	Mesure de réduction
Dépassement des quotas de l'API GitHub	Moyenne	Moyen	Cache, requêtes conditionnelles (ETag), webhooks plutôt qu'interrogations répétées.
Indisponibilité de GitHub	Faible	Élevé	File d'attente RabbitMQ, reprises automatiques, lecture de secours depuis PostgreSQL.
Fuite de données personnelles dans les dépôts	Moyenne	Élevé	Dépôts privés, données fictives, jeton minimal, chiffrement des champs sensibles.
Désynchronisation entre GitHub et PostgreSQL	Moyenne	Moyen	GitHub comme source de vérité, webhooks idempotents, réconciliation périodique.
Conflits de fusion	Moyenne	Faible	Une branche et un dossier par acte, un seul agent assigné à la fois.
Complexité de l'architecture répartie	Élevée	Moyen	Prototype minimal d'abord, ajout progressif des services, Docker Compose pour l'intégration.
Retard sur le planning	Moyenne	Moyen	Priorisation des besoins (MVP), points d'avancement hebdomadaires, tâches suivies dans GitHub Projects.

11. Livrables et critères de recette

11.1 Livrables attendus

- Le présent **cahier des charges** (formats Word et PDF).
- Le **dossier de conception** : diagrammes UML (cas d'utilisation, classes, séquence, composants, déploiement) et maquettes.
- Le **code source** hébergé dans un dépôt GitHub : dossier frontend (React) et dossier services (Python).
- L'**application déployable** par la commande docker compose up, avec un jeu de données de démonstration.
- La **documentation** : README, documentation OpenAPI, guide utilisateur et guide d'installation.
- Le **rapport de tests**.

11.2 Critères de recette

Scénario de recette	Résultat attendu
Un citoyen crée un compte et dépose une demande d'attestation de résidence	La demande apparaît dans son suivi, une Issue GitHub est créée et un e-mail de confirmation est reçu.

Scénario de recette	Résultat attendu
Un agent prend en charge la demande, modifie le brouillon et le soumet	Une branche et des commits existent dans GitHub ; une Pull Request est ouverte ; l'état passe à « En validation ».
Un validateur demande des corrections puis approuve	Les revues apparaissent dans la PR ; les états évoluent conformément au workflow.
Le validateur signe l'acte	La PR est fusionnée, le PDF avec QR code est généré, le tag est posé, le citoyen peut télécharger l'acte.
Un utilisateur tente une action non autorisée pour son rôle	L'accès est refusé (code 403) et l'événement est journalisé.
GitHub est rendu indisponible pendant le dépôt d'une demande	La demande n'est pas perdue et est transmise à GitHub au retour du service.
L'historique d'un acte est consulté	La liste des commits correspond exactement aux actions effectuées, avec auteurs et dates.
Le pipeline d'intégration continue est exécuté	Lint, tests et construction des images réussissent.

12. Conclusion

GestActes tire parti de la puissance de GitHub (historique, revue, protection de branche, automatisation) pour répondre aux exigences de traçabilité et de validation des actes administratifs, tout en offrant un terrain concret pour appliquer les principes d'une architecture répartie : découpage en services, communication synchrone et asynchrone, résilience et sécurité.

La démarche proposée est progressive : un prototype minimal de la passerelle GitHub permet de valider très tôt le principe, puis les services et les interfaces sont ajoutés par itérations. La version 1 se concentre sur trois types d'actes et sur les besoins de priorité Haute, ce qui garantit un livrable complet et démontrable dans les délais.

Annexe A : écriture d'un fichier dans GitHub

Extrait de la passerelle GitHub : écriture d'un fichier dans le dépôt de données via l'API REST (endpoint Contents). C'est le prototype minimal recommandé pour débuter.

```
import base64, httpx
from os import environ

GITHUB_API = "https://api.github.com"
HEADERS = {
    "Authorization": f"Bearer {environ['GITHUB_TOKEN']}",
    "Accept": "application/vnd.github+json",
}

async def creer_fichier(depot, chemin, contenu, message, branche="main"):
    url = f"{GITHUB_API}/repos/{depot}/contents/{chemin}"
    corps = {
        "message": message,
        "content": base64.b64encode(contenu.encode()).decode(),
        "branch": branche,
    }
```

```
    async with httpx.AsyncClient(timeout=10) as client:
        r = await client.put(url, json=corps, headers=HEADERS)
        r.raise_for_status()
        return r.json()["commit"]["sha"]
```

Annexe B : exemple de workflow GitHub Actions

Génération automatique du PDF lorsqu'un acte est fusionné dans la branche principale.

```
name: generer-pdf
on:
  push:
    branches: [main]
    paths: ["actes/**/acte.json"]
jobs:
  pdf:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install reportlab qrcode[pil]
      - run: python scripts/generer_pdf.py
      - name: Commit du PDF et tag
        run: |
          git config user.name "gestactes-bot"
          git config user.email "bot@gestactes.local"
          git add actes/**/acte.pdf
          git commit -m "chore: generation du PDF" || true
          git push
```

Annexe C : structure proposée du code source

```
gestactes/
  docker-compose.yml
  .env.example
  frontend/                  # application React (Vite, TypeScript)
    src/
      pages/  components/  api/  hooks/  routes/
  services/
    gateway/                 # API Gateway FastAPI
    auth/                    # service Authentification
    actes/                   # service Actes
    workflow/                # service Workflow + webhooks
    github_gateway/          # passerelle GitHub
    notification/            # service Notification
  docs/                      # cahier des charges, UML, maquettes
  .github/workflows/         # intégration continue
```